Phase 3 Lab Report

Group 22

Jamie Bell, Nolan Steed, Hayden Richardson

20259237, 20295240, 20404771

March 31st, 2026

“We do hereby verify that this written lab report is our own work and contains our own original ideas, concepts, and designs. No portion of this report has been copied in whole or in part from another source, with the possible exception of properly referenced material"

Table of Contents

Table of Contents 2

Verilog Logic Code 2

Control_Unit.v 2

CPU.v 19

RAM.v 27

Select_and_encode.v 30

Datapath.v 32

Verilog Testbench Code 42

Phase3_tb.v 42

Simulation 46

Verilog Logic Code

Control_Unit.v

```
// RAM (512 x 32) code
`timescale 1ns/10ps


module control_unit(
  input Clock,
  input Reset,
  input Stop,
  input CON_FF,
  input [31:0] IR,

  output reg [15:0] Rin,
  output reg [15:0] Rout,
  output reg Gra,
```

```
output reg Grb,
output reg Grc,
output reg BAout,
output reg CONin,

output reg PCout,
output reg PCin,
output reg IRin,
output reg IRout,
output reg MARin,
output reg MDRin,
output reg MDRout,
output reg HIin,
output reg HIout,
output reg LOin,
output reg LOout,
output reg Yin,
output reg Zin,
output reg Zhighout,
output reg Zlowout,
output reg InPortout,
output reg OutPortin,
output reg Cout,
output reg IncPC,
output reg Read,
output reg Write,
output reg [4:0] ALUop,
```

```
    output reg Run,
    output reg [7:0] state_dbg
);

localparam S_RESET     = 8'd0;
localparam S_FETCH0    = 8'd1;
localparam S_FETCH1    = 8'd2;
localparam S_FETCH2    = 8'd3;

localparam S_DECODE    = 8'd4;

localparam S_LDI3      = 8'd10;
localparam S_LDI4      = 8'd11;
localparam S_LDI5      = 8'd12;

localparam S_LD3       = 8'd20;
localparam S_LD4       = 8'd21;
localparam S_LD5       = 8'd22;
localparam S_LD6       = 8'd23;
localparam S_LD7       = 8'd24;

localparam S_ST3       = 8'd30;
localparam S_ST4       = 8'd31;
localparam S_ST5       = 8'd32;
localparam S_ST6       = 8'd33;
localparam S_ST7       = 8'd34;
```

```
localparam S_R3       = 8'd40;
localparam S_R4       = 8'd41;
localparam S_R5       = 8'd42;

localparam S_I3       = 8'd50;
localparam S_I4       = 8'd51;
localparam S_I5       = 8'd52;

localparam S_U3       = 8'd60;
localparam S_U4       = 8'd61;
localparam S_U5       = 8'd62;

localparam S_MULDIV3   = 8'd70;
localparam S_MULDIV4   = 8'd71;
localparam S_MULDIV5   = 8'd72;
localparam S_MULDIV6   = 8'd73;

localparam S_MFHI3     = 8'd80;
localparam S_MFFLO3    = 8'd81;

localparam S_BRANCH3   = 8'd90;
localparam S_BRANCH4   = 8'd91;
localparam S_BRANCH5   = 8'd92;
localparam S_BRANCH6   = 8'd93;

localparam S_JR3      = 8'd100;
localparam S_JAL3     = 8'd101;
```

```
localparam S_JAL4      = 8'd102;


localparam S_HALT      = 8'd110;


reg [7:0] present_state, next_state;


wire [4:0] opcode = IR[31:27];


always @(posedge Clock or posedge Reset) begin
   if (Reset)
      present_state <= S_RESET;
   else
      present_state <= next_state;
end


always @(*) begin
   next_state = present_state;


   case (present_state)
      S_RESET:  next_state = S_FETCH0;
      S_FETCH0: next_state = S_FETCH1;
                                 S_FETCH1: next_state = S_FETCH2;
                                 S_FETCH2: next_state = S_DECODE;


                                 S_DECODE: begin
                                              case (opcode)
                                                           5'b10001: next_state = S_LDI3;
```

```
5'b10000: next_state = S_LD3;
5'b10010: next_state = S_ST3;

5'b00000,
5'b00001,
5'b00010,
5'b00011,
5'b00100,
5'b00101,
5'b00110,
5'b00111,
5'b01000: next_state = S_R3;

5'b01001,
5'b01010,
5'b01011: next_state = S_I3;

5'b01110,
5'b01111: next_state = S_U3;

5'b01100,
5'b01101: next_state = S_MULDIV3;

5'b11000: next_state = S_MFHI3;
5'b11001: next_state = S_MFFLO3;
5'b10101: next_state = S_BRANCH3;
5'b10100: next_state = S_JR3;
```

```
                                        5'b10011: next_state = S_JAL3;
                                        5'b11010: next_state = S_FETCH0;
                                        5'b11011: next_state = S_HALT;
                                        default:  next_state = S_FETCH0;
                                endcase
                        end

S_LDI3: next_state = S_LDI4;
S_LDI4: next_state = S_LDI5;
S_LDI5: next_state = S_FETCH0;

S_LD3: next_state = S_LD4;
S_LD4: next_state = S_LD5;
S_LD5: next_state = S_LD6;
S_LD6: next_state = S_LD7;
S_LD7: next_state = S_FETCH0;

S_ST3: next_state = S_ST4;
S_ST4: next_state = S_ST5;
S_ST5: next_state = S_ST6;
S_ST6: next_state = S_ST7;
S_ST7: next_state = S_FETCH0;

S_R3: next_state = S_R4;
S_R4: next_state = S_R5;
S_R5: next_state = S_FETCH0;
```

```
S_I3: next_state = S_I4;
S_I4: next_state = S_I5;
S_I5: next_state = S_FETCH0;

S_U3: next_state = S_U4;
S_U4: next_state = S_U5;
S_U5: next_state = S_FETCH0;

S_MULDIV3: next_state = S_MULDIV4;
S_MULDIV4: next_state = S_MULDIV5;
S_MULDIV5: next_state = S_MULDIV6;
S_MULDIV6: next_state = S_FETCH0;

S_MFHI3: next_state = S_FETCH0;
S_MFFLO3: next_state = S_FETCH0;

S_BRANCH3: next_state = S_BRANCH4;
S_BRANCH4: next_state = S_BRANCH5;
S_BRANCH5: next_state = S_BRANCH6;
S_BRANCH6: next_state = S_FETCH0;

S_JR3: next_state = S_FETCH0;
S_JAL3: next_state = S_JAL4;
S_JAL4: next_state = S_FETCH0;

S_HALT: next_state = S_HALT;
default: next_state = S_FETCH0;
```

```
    endcase

    if (Stop)
        next_state = S_HALT;
end

always @(*) begin
    Rin      = 16'b0;
    Rout     = 16'b0;
    Gra      = 1'b0;
    Grb      = 1'b0;
    Grc      = 1'b0;
    BAout    = 1'b0;
    CONin    = 1'b0;
    PCout    = 1'b0;
    PCin     = 1'b0;
    IRin     = 1'b0;
    IRout    = 1'b0;
    MARin    = 1'b0;
    MDRin    = 1'b0;
    MDRout   = 1'b0;
    HIin     = 1'b0;
    HIout    = 1'b0;
    LOin     = 1'b0;
    LOout    = 1'b0;
    Yin      = 1'b0;
    Zin      = 1'b0;
```

```
Zhighout  = 1'b0;
Zlowout   = 1'b0;
InPortout = 1'b0;
OutPortin = 1'b0;
Cout      = 1'b0;
IncPC     = 1'b0;
Read      = 1'b0;
Write     = 1'b0;
ALUop     = 5'b00000;
Run       = (present_state != S_HALT);
state_dbg = present_state;

case (present_state)
  S_FETCH0: begin
    PCout = 1'b1;
    MARin = 1'b1;
    IncPC = 1'b1;
    Zin   = 1'b1;
  end

  S_FETCH1: begin
    Zlowout = 1'b1;
    PCin    = 1'b1;
    Read    = 1'b1;
    MDRin   = 1'b1;
  end
```

```
S_FETCH2: begin
   MDRout = 1'b1;
   IRin   = 1'b1;
end

S_LDI3: begin
   Grb   = 1'b1;
   BAout = 1'b1;
   Yin   = 1'b1;
end
S_LDI4: begin
   Cout  = 1'b1;
   ALUop = 5'b00000;
   Zin   = 1'b1;
end
S_LDI5: begin
   Zlowout = 1'b1;
   Gra     = 1'b1;
   Rin     = 16'hFFFF;
end

S_LD3: begin
   Grb   = 1'b1;
   BAout = 1'b1;
   Yin   = 1'b1;
end
S_LD4: begin
```

```
    Cout  = 1'b1;
    ALUop = 5'b00000;
    Zin   = 1'b1;
  end
  S_LD5: begin
    Zlowout = 1'b1;
    MARin   = 1'b1;
  end
  S_LD6: begin
    Read  = 1'b1;
    MDRin = 1'b1;
  end
  S_LD7: begin
    MDRout = 1'b1;
    Gra    = 1'b1;
    Rin    = 16'hFFFF;
  end

  S_ST3: begin
    Grb   = 1'b1;
    BAout = 1'b1;
    Yin   = 1'b1;
  end
  S_ST4: begin
    Cout  = 1'b1;
    ALUop = 5'b00000;
    Zin   = 1'b1;
```

```
end
S_ST5: begin
   Zlowout = 1'b1;
   MARin   = 1'b1;
end
S_ST6: begin
   Gra    = 1'b1;
   Rout   = 16'hFFFF;
   MDRin  = 1'b1;
end
S_ST7: begin
   Write = 1'b1;
end

S_R3: begin
   Grb  = 1'b1;
   Rout = 16'hFFFF;
   Yin  = 1'b1;
end
S_R4: begin
   Grc   = 1'b1;
   Rout  = 16'hFFFF;
   ALUop = opcode;
   Zin   = 1'b1;
end
S_R5: begin
   Zlowout = 1'b1;
```

```
    Gra    = 1'b1;
    Rin    = 16'hFFFF;
  end

  S_I3: begin
    Grb  = 1'b1;
    Rout = 16'hFFFF;
    Yin  = 1'b1;
  end
  S_I4: begin
    Cout = 1'b1;
    Zin  = 1'b1;
    case (opcode)
      5'b01001: ALUop = 5'b00000;
      5'b01010: ALUop = 5'b00010;
      5'b01011: ALUop = 5'b00011;
      default:  ALUop = 5'b00000;
    endcase
  end
  S_I5: begin
    Zlowout = 1'b1;
    Gra    = 1'b1;
    Rin    = 16'hFFFF;
  end

  S_U3: begin
    Grb  = 1'b1;
```

```
    Rout = 16'hFFFF;
    Yin  = 1'b1;
  end
  S_U4: begin
    ALUop = opcode;
    Zin   = 1'b1;
  end
  S_U5: begin
    Zlowout = 1'b1;
    Gra     = 1'b1;
    Rin     = 16'hFFFF;
  end

  S_MULDIV3: begin
    Gra  = 1'b1;
    Rout = 16'hFFFF;
    Yin  = 1'b1;
  end
  S_MULDIV4: begin
    Grb   = 1'b1;
    Rout  = 16'hFFFF;
    ALUop = opcode;
    Zin   = 1'b1;
  end
  S_MULDIV5: begin
    Zlowout = 1'b1;
    LOin    = 1'b1;
```

```
end
S_MULDIV6: begin
   Zhighout = 1'b1;
   HIin    = 1'b1;
end

S_MFHI3: begin
   HIout = 1'b1;
   Gra   = 1'b1;
   Rin   = 16'hFFFF;
end
S_MFFLO3: begin
   LOout = 1'b1;
   Gra   = 1'b1;
   Rin   = 16'hFFFF;
end

S_BRANCH3: begin
   Gra   = 1'b1;
   Rout  = 16'hFFFF;
   CONin = 1'b1;
end
S_BRANCH4: begin
   PCout = 1'b1;
   Yin   = 1'b1;
end
S_BRANCH5: begin
```

```
    Cout  = 1'b1;
    ALUop = 5'b00000;
    Zin   = 1'b1;
  end
  S_BRANCH6: begin
    if (CON_FF) begin
      Zlowout = 1'b1;
      PCin    = 1'b1;
    end
  end

  S_JR3: begin
    Gra  = 1'b1;
    Rout = 16'hFFFF;
    PCin = 1'b1;
  end

  S_JAL3: begin
    PCout = 1'b1;
    Rin   = 16'h1000;
  end
  S_JAL4: begin
    Gra  = 1'b1;
    Rout = 16'hFFFF;
    PCin = 1'b1;
  end
```

```
      S_HALT: begin
         Run = 1'b0;
      end
   endcase

   if (Stop)
      Run = 1'b0;
end

endmodule
```

CPU.v

```
// MAR code
`timescale 1ns/10ps

module CPU(
   input Clock,
   input Reset,
   input Stop,
   input [31:0] InPortData,

   output Run,
   output [7:0] state_dbg,
   output [31:0] IR_dbg,
   output [31:0] PC_dbg,
   output [31:0] MDR_dbg,
   output [31:0] MAR_dbg,
```

```
output [31:0] HI_dbg,
output [31:0] LO_dbg,
output [31:0] R0_dbg,
output [31:0] R1_dbg,
output [31:0] R2_dbg,
output [31:0] R3_dbg,
output [31:0] R4_dbg,
output [31:0] R5_dbg,
output [31:0] R6_dbg,
output [31:0] R7_dbg,
output [31:0] R8_dbg,
output [31:0] R9_dbg,
output [31:0] R10_dbg,
output [31:0] R11_dbg,
output [31:0] R12_dbg,
output [31:0] R13_dbg,
output [31:0] R14_dbg,
output [31:0] R15_dbg,
output [31:0] OutPortData,
output [31:0] RAMDataOut,
output [31:0] Mem89_dbg,
output [31:0] MemA3_dbg,
output CON_dbg,

output [15:0] Rin_dbg,
output [15:0] Rout_dbg,
output Gra_dbg,
```

```
    output Grb_dbg,
    output Grc_dbg,
    output BAout_dbg,
    output CONin_dbg,
    output PCout_dbg,
    output PCin_dbg,
    output IRin_dbg,
    output IRout_dbg,
    output MARin_dbg,
    output MDRin_dbg,
    output MDRout_dbg,
    output HIin_dbg,
    output HIout_dbg,
    output LOin_dbg,
    output LOout_dbg,
    output Yin_dbg,
    output Zin_dbg,
    output Zhighout_dbg,
    output Zlowout_dbg,
    output InPortout_dbg,
    output OutPortin_dbg,
    output Cout_dbg,
    output IncPC_dbg,
    output Read_dbg,
    output Write_dbg,
    output [4:0] ALUop_dbg
);
```

```
wire [15:0] Rin;
wire [15:0] Rout;
wire Gra, Grb, Grc, BAout, CONin;
wire PCout, PCin, IRin, IRout, MARin, MDRin, MDRout;
wire HIin, HIout, LOin, LOout, Yin, Zin, Zhighout, Zlowout;
wire InPortout, OutPortin, Cout, IncPC, Read, Write;
wire [4:0] ALUop;
wire CON;

control_unit CU0(
   .Clock(Clock),
   .Reset(Reset),
   .Stop(Stop),
   .CON_FF(CON),
   .IR(IR_dbg),
   .Rin(Rin),
   .Rout(Rout),
   .Gra(Gra),
   .Grb(Grb),
   .Grc(Grc),
   .BAout(BAout),
   .CONin(CONin),
   .PCout(PCout),
   .PCin(PCin),
   .IRin(IRin),
   .IRout(IRout),
```

```
    .MARin(MARin),
    .MDRin(MDRin),
    .MDRout(MDRout),
    .HIin(HIin),
    .HIout(HIout),
    .LOin(LOin),
    .LOout(LOout),
    .Yin(Yin),
    .Zin(Zin),
    .Zhighout(Zhighout),
    .Zlowout(Zlowout),
    .InPortout(InPortout),
    .OutPortin(OutPortin),
    .Cout(Cout),
    .IncPC(IncPC),
    .Read(Read),
    .Write(Write),
    .ALUop(ALUop),
    .Run(Run),
    .state_dbg(state_dbg)
);

assign Rin_dbg = Rin;
assign Rout_dbg = Rout;
assign Gra_dbg = Gra;
assign Grb_dbg = Grb;
assign Grc_dbg = Grc;
```

```
assign BAout_dbg = BAout;
assign CONin_dbg = CONin;
assign PCout_dbg = PCout;
assign PCin_dbg = PCin;
assign IRin_dbg = IRin;
assign IRout_dbg = IRout;
assign MARin_dbg = MARin;
assign MDRin_dbg = MDRin;
assign MDRout_dbg = MDRout;
assign HIin_dbg = HIin;
assign HIout_dbg = HIout;
assign LOin_dbg = LOin;
assign LOout_dbg = LOout;
assign Yin_dbg = Yin;
assign Zin_dbg = Zin;
assign Zhighout_dbg = Zhighout;
assign Zlowout_dbg = Zlowout;
assign InPortout_dbg = InPortout;
assign OutPortin_dbg = OutPortin;
assign Cout_dbg = Cout;
assign IncPC_dbg = IncPC;
assign Read_dbg = Read;
assign Write_dbg = Write;
assign ALUop_dbg = ALUop;
assign CON_dbg = CON;

datapath DP0(
```

```
.Reset(Reset),
.Rin(Rin),
.Rout(Rout),
.Gra(Gra),
.Grb(Grb),
.Grc(Grc),
.BAout(BAout),
.CONin(CONin),
.CON(CON),
.PCout(PCout),
.PCin(PCin),
.IRin(IRin),
.IRout(IRout),
.MARin(MARin),
.MDRin(MDRin),
.MDRout(MDRout),
.HIin(HIin),
.HIout(HIout),
.LOin(LOin),
.LOout(LOout),
.Yin(Yin),
.Zin(Zin),
.Zhighout(Zhighout),
.Zlowout(Zlowout),
.InPortout(InPortout),
.OutPortin(OutPortin),
.Cout(Cout),
```

```
.IncPC(IncPC),
.Read(Read),
.Write(Write),
.opcode(ALUop),
.Clock(Clock),
.Mdatain(32'b0),
.InPortData(InPortData),
.OutPortData(OutPortData),
.MARData(MAR_dbg),
.RAMDataOut(RAMDataOut),
.IRData(IR_dbg),
.PCData(PC_dbg),
.MDRData(MDR_dbg),
.HIData(HI_dbg),
.LOData(LO_dbg),
.R0Data(R0_dbg),
.R1Data(R1_dbg),
.R2Data(R2_dbg),
.R3Data(R3_dbg),
.R4Data(R4_dbg),
.R5Data(R5_dbg),
.R6Data(R6_dbg),
.R7Data(R7_dbg),
.R8Data(R8_dbg),
.R9Data(R9_dbg),
.R10Data(R10_dbg),
.R11Data(R11_dbg),
```

```
   .R12Data(R12_dbg),
   .R13Data(R13_dbg),
   .R14Data(R14_dbg),
   .R15Data(R15_dbg),
   .Mem89Data(Mem89_dbg),
   .MemA3Data(MemA3_dbg)
);

endmodule
```

RAM.v

```
//RAM code
//Group: 22
module RAM(
   input clock,
   input Read,
   input Write,
   input [8:0] Address,
   input [31:0] DataIn,
   output reg [31:0] DataOut,
   output [31:0] Mem89,
   output [31:0] MemA3
);

reg [31:0] mem [0:511];
integer i;
```

```
initial begin
  for (i = 0; i < 512; i = i + 1)
    mem[i] = 32'b0;

  mem[9'h089] = 32'h000000A7;
  mem[9'h0A3] = 32'h00000068;

  mem[9'h000] = 32'h8A800043;
  mem[9'h001] = 32'h8AA80006;
  mem[9'h002] = 32'h82000089;
  mem[9'h003] = 32'h8A200004;
  mem[9'h004] = 32'h8027FFF8;
  mem[9'h005] = 32'h89000004;
  mem[9'h006] = 32'h8A800087;
  mem[9'h007] = 32'hAA980003;
  mem[9'h008] = 32'h8AA80005;
  mem[9'h009] = 32'h80AFFFFD;
  mem[9'h00A] = 32'hD0000000;
  mem[9'h00B] = 32'hA8900002;
  mem[9'h00C] = 32'h89A80007;
  mem[9'h00D] = 32'h8B9FFFFC;
  mem[9'h00E] = 32'h03A90000;
  mem[9'h00F] = 32'h48880003;
  mem[9'h010] = 32'h70880000;
  mem[9'h011] = 32'h78880000;
  mem[9'h012] = 32'h5088000F;
  mem[9'h013] = 32'h3A010000;
```

```
mem[9'h014] = 32'h58A00005;
mem[9'h015] = 32'h2A090000;
mem[9'h016] = 32'h22A90000;
mem[9'h017] = 32'h928000A3;
mem[9'h018] = 32'h42810000;
mem[9'h019] = 32'h1B900000;
mem[9'h01A] = 32'h12280000;
mem[9'h01B] = 32'h93A00089;
mem[9'h01C] = 32'h082B8000;
mem[9'h01D] = 32'h32290000;
mem[9'h01E] = 32'h8B800007;
mem[9'h01F] = 32'h89800019;
mem[9'h020] = 32'h69B80000;
mem[9'h021] = 32'hC0800000;
mem[9'h022] = 32'hCB000000;
mem[9'h023] = 32'h61B80000;
mem[9'h024] = 32'h8C380002;
mem[9'h025] = 32'h8C9FFFFC;
mem[9'h026] = 32'h8D300003;
mem[9'h027] = 32'h8D880005;
mem[9'h028] = 32'h9D000000;
mem[9'h029] = 32'hD8000000;

mem[9'h0B2] = 32'h07450000;
mem[9'h0B3] = 32'h0ECD8000;
mem[9'h0B4] = 32'h0F768000;
mem[9'h0B5] = 32'hA6000000;
```

```
end


always @(posedge clock) begin
   if (Write)
      mem[Address] <= DataIn;
end


always @(*) begin
   if (Read)
      DataOut = mem[Address];
   else
      DataOut = 32'b0;
end


assign Mem89 = mem[9'h089];
assign MemA3 = mem[9'h0A3];


endmodule
```

Select_and_encode.v

```
// Select_Encode code
// Group: 22


module select_encode(
   input [31:0] IR,
   input Gra, Grb, Grc,
   input Rin, Rout,
   output reg [15:0] Rin_out,
```

```
    output reg [15:0] Rout_out
);

    wire [3:0] Ra = IR[26:23];
    wire [3:0] Rb = IR[22:19];
    wire [3:0] Rc = IR[18:15];

    reg [3:0] Sel;

    always @(*) begin
        if (Gra) Sel = Ra;
        else if (Grb) Sel = Rb;
        else if (Grc) Sel = Rc;
        else Sel = 4'b0000;
    end

    always @(*) begin
        Rin_out = 16'b0;
        Rout_out = 16'b0;

        if (Rin)
            Rin_out[Sel] = 1'b1;

        if (Rout)
            Rout_out[Sel] = 1'b1;
    end
```

```
endmodule
```

Datapath.v

```
`timescale 1ns/10ps

//Datapath.v code

//Group: 22


module datapath (

   input Reset,

   input  [15:0] Rin,

   input  [15:0] Rout,

   input Gra, Grb, Grc,

   input BAout,

   input CONin,

   output reg CON,

   input PCout,

   input PCin,

   input IRin,

   input IRout,

   input MARin,

   input MDRin,

   input MDRout,

   input HIin,

   input HIout,

   input LOin,

   input LOout,

   input Yin,

   input Zin,
```

```
input Zhighout,
input Zlowout,
input InPortout,
input OutPortin,
input Cout,
input IncPC,
input Read,
input Write,
input [4:0] opcode,
input Clock,
input [31:0] Mdatain,
input [31:0] InPortData,
output [31:0] OutPortData,
output [31:0] MARData,
output [31:0] RAMDataOut,
output [31:0] IRData,
output [31:0] PCData,
output [31:0] MDRData,
output [31:0] HIData,
output [31:0] LOData,
output [31:0] R0Data,
output [31:0] R1Data,
output [31:0] R2Data,
output [31:0] R3Data,
output [31:0] R4Data,
output [31:0] R5Data,
output [31:0] R6Data,
```

```
    output [31:0] R7Data,
    output [31:0] R8Data,
    output [31:0] R9Data,
    output [31:0] R10Data,
    output [31:0] R11Data,
    output [31:0] R12Data,
    output [31:0] R13Data,
    output [31:0] R14Data,
    output [31:0] R15Data,
    output [31:0] Mem89Data,
    output [31:0] MemA3Data
);

    reg clear_int;
    initial begin
        clear_int = 1'b1;
        #5 clear_int = 1'b0;
    end

    wire clear_all = clear_int | Reset;
    wire [31:0] BusMuxOut;

    wire [31:0] R0_q,  R1_q,  R2_q,  R3_q;
    wire [31:0] R4_q,  R5_q,  R6_q,  R7_q;
    wire [31:0] R8_q,  R9_q,  R10_q, R11_q;
    wire [31:0] R12_q, R13_q, R14_q, R15_q;
```

```
wire [31:0] IR_q;
wire [3:0] Ra = IR_q[26:23];
wire [3:0] Rb = IR_q[22:19];
wire [3:0] Rc = IR_q[18:15];

wire [15:0] Rin_sel;
wire [15:0] Rout_sel;
reg [15:0] Rin_decoded;
reg [15:0] Rout_decoded;

select_encode SE0(
   .IR(IR_q),
   .Gra(Gra),
   .Grb(Grb),
   .Grc(Grc),
   .Rin(|Rin),
   .Rout((|Rout) || BAout),
   .Rin_out(Rin_sel),
   .Rout_out(Rout_sel)
);

always @(*) begin
   if (Gra || Grb || Grc) begin
      Rin_decoded = Rin_sel;
      Rout_decoded = Rout_sel;
   end
   else begin
```

```
      Rin_decoded = Rin;
      Rout_decoded = Rout;
    end
  end

  register #(32,32,32'h0) R0  (clear_all, Clock, Rin_decoded[0],  BusMuxOut, R0_q);
  register #(32,32,32'h0) R1  (clear_all, Clock, Rin_decoded[1],  BusMuxOut, R1_q);
  register #(32,32,32'h0) R2  (clear_all, Clock, Rin_decoded[2],  BusMuxOut, R2_q);
  register #(32,32,32'h0) R3  (clear_all, Clock, Rin_decoded[3],  BusMuxOut, R3_q);
  register #(32,32,32'h0) R4  (clear_all, Clock, Rin_decoded[4],  BusMuxOut, R4_q);
  register #(32,32,32'h0) R5  (clear_all, Clock, Rin_decoded[5],  BusMuxOut, R5_q);
  register #(32,32,32'h0) R6  (clear_all, Clock, Rin_decoded[6],  BusMuxOut, R6_q);
  register #(32,32,32'h0) R7  (clear_all, Clock, Rin_decoded[7],  BusMuxOut, R7_q);
  register #(32,32,32'h0) R8  (clear_all, Clock, Rin_decoded[8],  BusMuxOut, R8_q);
  register #(32,32,32'h0) R9  (clear_all, Clock, Rin_decoded[9],  BusMuxOut, R9_q);
  register #(32,32,32'h0) R10 (clear_all, Clock, Rin_decoded[10], BusMuxOut, R10_q);
  register #(32,32,32'h0) R11 (clear_all, Clock, Rin_decoded[11], BusMuxOut, R11_q);
  register #(32,32,32'h0) R12 (clear_all, Clock, Rin_decoded[12], BusMuxOut, R12_q);
  register #(32,32,32'h0) R13 (clear_all, Clock, Rin_decoded[13], BusMuxOut, R13_q);
  register #(32,32,32'h0) R14 (clear_all, Clock, Rin_decoded[14], BusMuxOut, R14_q);
  register #(32,32,32'h0) R15 (clear_all, Clock, Rin_decoded[15], BusMuxOut, R15_q);

  wire [31:0] R0_modified = (BAout && Rout_decoded[0]) ? 32'b0 : R0_q;

  wire [31:0] MAR_q;
  wire [31:0] MDR_q;
  wire [31:0] HI_q, LO_q;
```

```
wire [31:0] Y_q;
wire [63:0] Z_q;
wire [31:0] InPort_q;
wire [31:0] OutPort_q;
wire [31:0] PC_q;

register #(32,32,32'h0) PC (clear_all, Clock, PCin, BusMuxOut, PC_q);
IR  IR0  (clear_all, Clock, IRin,  BusMuxOut, IR_q);
MAR MAR0 (clear_all, Clock, MARin, BusMuxOut, MAR_q);

wire [31:0] RAM_q;
wire [31:0] Mem89_q;
wire [31:0] MemA3_q;
RAM RAM0 (
   .clock(Clock),
   .Read(Read),
   .Write(Write),
   .Address(MAR_q[8:0]),
   .DataIn(MDR_q),
   .DataOut(RAM_q),
   .Mem89(Mem89_q),
   .MemA3(MemA3_q)
);

wire [31:0] MDR_mem_input = (Mdatain != 32'b0) ? Mdatain : RAM_q;

MDR MDR0 (clear_all, Clock, MDRin, Read, BusMuxOut, MDR_mem_input, MDR_q);
```

```
register #(32,32,32'h0) HI (clear_all, Clock, HIin, BusMuxOut, HI_q);
register #(32,32,32'h0) LO (clear_all, Clock, LOin, BusMuxOut, LO_q);
register #(32,32,32'h0) Y  (clear_all, Clock, Yin,  BusMuxOut, Y_q);

InPort INPORT0 (
   .clear(clear_all),
   .clock(Clock),
   .ExternalInput(InPortData),
   .BusMuxIn_InPort(InPort_q)
);

OutPort OUTPORT0 (
   .clear(clear_all),
   .clock(Clock),
   .OutPortin(OutPortin),
   .BusMuxOut(BusMuxOut),
   .OutPortData(OutPort_q)
);

wire CON_next;

con_ff CONFF0(
   .Bus(BusMuxOut),
   .IR(IR_q),
   .CON_out(CON_next)
);
```

```
always @(posedge Clock or posedge clear_all) begin
   if (clear_all)
      CON <= 1'b0;
   else if (CONin)
      CON <= CON_next;
end

wire [63:0] ALU_Z;
alu ALU0 (
   .A(Y_q),
   .B(BusMuxOut),
   .ALUop(opcode),
   .Z(ALU_Z)
);

wire [31:0] pc_plus_one;
wire pc_cout;
adder32 PCINC (
   .A(PC_q),
   .B(32'h00000001),
   .SUB(1'b0),
   .S(pc_plus_one),
   .Cout(pc_cout)
);

wire [63:0] Z_in = IncPC ? {32'b0, pc_plus_one} : ALU_Z;
register64 #(64,64,64'h0) Z (
```

```
    .clear(clear_all),
    .clock(Clock),
    .enable(Zin),
    .BusMuxOut(Z_in),
    .BusMuxIn(Z_q)
);

wire [31:0] Zhigh = Z_q[63:32];
wire [31:0] Zlow  = Z_q[31:0];
wire [31:0] C_sign_extended = {{13{IR_q[18]}}, IR_q[18:0]};

Bus BUS0 (
    R0_modified, R1_q, R2_q, R3_q,
    R4_q, R5_q, R6_q, R7_q,
    R8_q, R9_q, R10_q, R11_q,
    R12_q, R13_q, R14_q, R15_q,
    PC_q, IR_q, MDR_q, HI_q, LO_q, Zhigh, Zlow, InPort_q, C_sign_extended,
    Rout_decoded[0], Rout_decoded[1], Rout_decoded[2], Rout_decoded[3],
    Rout_decoded[4], Rout_decoded[5], Rout_decoded[6], Rout_decoded[7],
    Rout_decoded[8], Rout_decoded[9], Rout_decoded[10], Rout_decoded[11],
    Rout_decoded[12], Rout_decoded[13], Rout_decoded[14], Rout_decoded[15],
    PCout, IRout, MDRout, HIout, LOout, Zhighout, Zlowout, InPortout, Cout,
    BusMuxOut
);

assign OutPortData = OutPort_q;
assign MARData = MAR_q;
```

```
    assign RAMDataOut = RAM_q;
    assign IRData = IR_q;
    assign PCData = PC_q;
    assign MDRData = MDR_q;
    assign HIData = HI_q;
    assign LOData = LO_q;
    assign R0Data = R0_q;
    assign R1Data = R1_q;
    assign R2Data = R2_q;
    assign R3Data = R3_q;
    assign R4Data = R4_q;
    assign R5Data = R5_q;
    assign R6Data = R6_q;
    assign R7Data = R7_q;
    assign R8Data = R8_q;
    assign R9Data = R9_q;
    assign R10Data = R10_q;
    assign R11Data = R11_q;
    assign R12Data = R12_q;
    assign R13Data = R13_q;
    assign R14Data = R14_q;
    assign R15Data = R15_q;
    assign Mem89Data = Mem89_q;
    assign MemA3Data = MemA3_q;

endmodule
```

Verilog Testbench Code

The testbenches are all very similar in style, with the key difference being the 5-digit opcode that corresponds to each instruction (set in T4).

Phase3_tb.v

```
// Ld_Direct_tb.v

// Group: 22

 `timescale 1ns/10ps

module phase3_tb;


reg Clock;

reg Reset;

reg Stop;

reg [31:0] InPortData;


wire Run;


wire [31:0] IR;

wire [31:0] PC;

wire [31:0] MAR;

wire [31:0] MDR;


wire [31:0] R0;

wire [31:0] R1;

wire [31:0] R2;

wire [31:0] R3;

wire [31:0] R4;
```

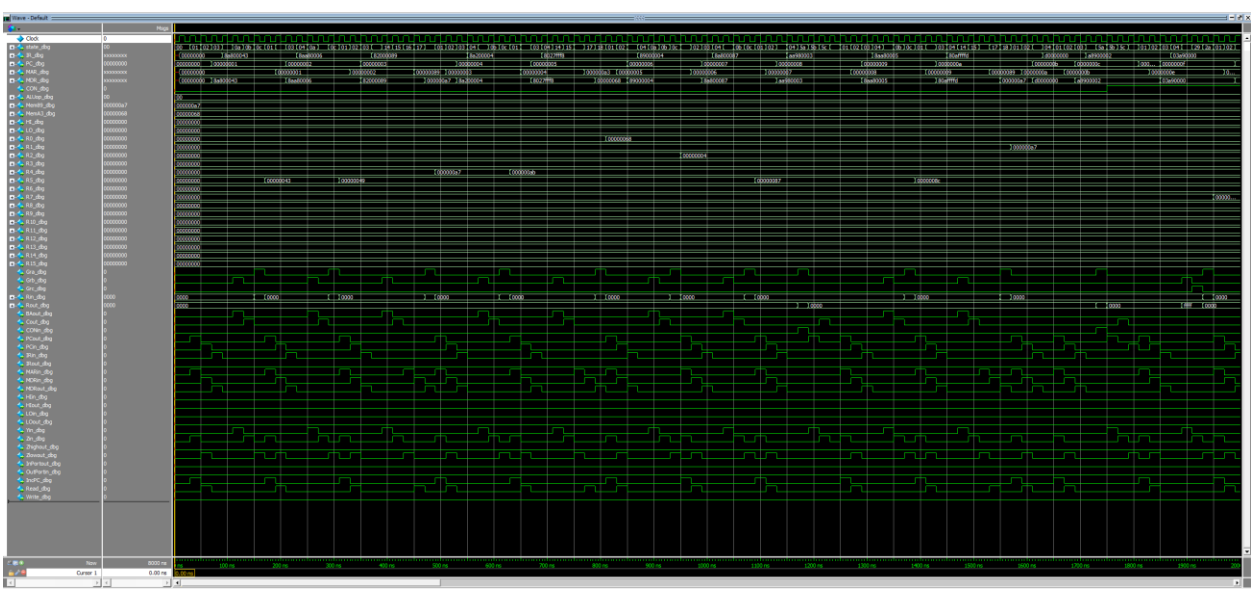
```
wire [31:0] R5;
wire [31:0] R6;
wire [31:0] R7;
wire [31:0] R8;
wire [31:0] R9;
wire [31:0] R10;
wire [31:0] R11;
wire [31:0] R12;
wire [31:0] R13;
wire [31:0] R14;
wire [31:0] R15;

wire [31:0] HI;
wire [31:0] LO;
wire [31:0] Zhigh;
wire [31:0] Zlow;

wire [31:0] mem_89;
wire [31:0] mem_A3;

wire PCout_dbg;
wire Zhighout_dbg;
wire Zlowout_dbg;
wire MDRout_dbg;
wire MARin_dbg;
wire PCin_dbg;
wire MDRin_dbg;
```

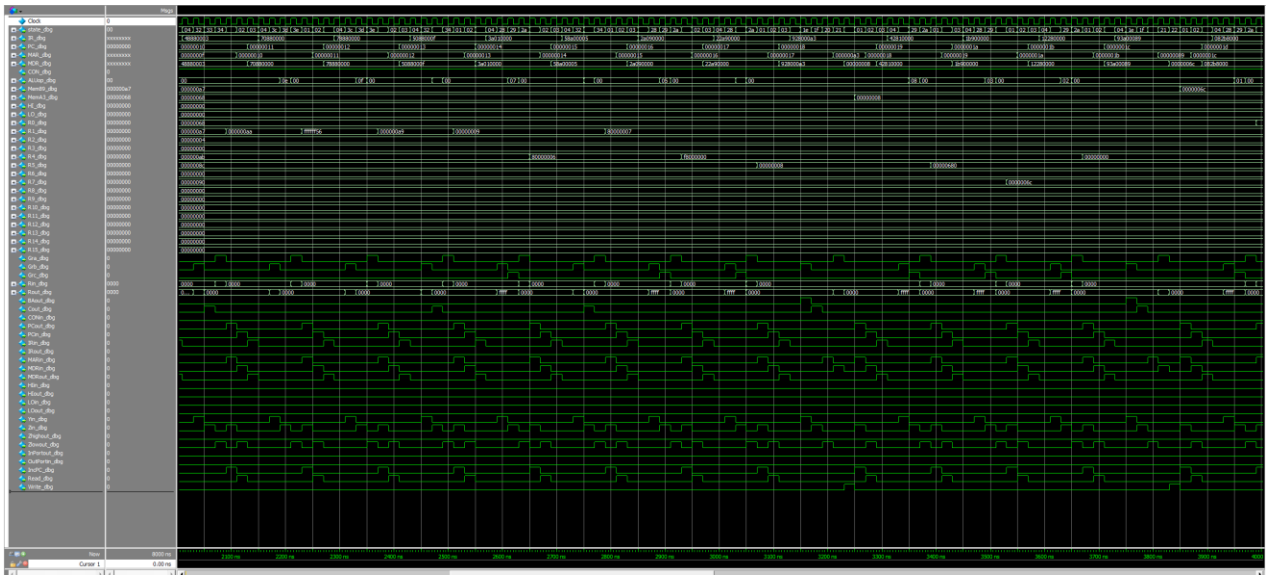
```
wire IRin_dbg;
wire Yin_dbg;
wire Zin_dbg;
wire HIin_dbg;
wire LOin_dbg;
wire Cout_dbg;
wire InPortout_dbg;
wire OutPortin_dbg;
wire Read_dbg;
wire Write_dbg;
wire [4:0] ALUop_dbg;

cpu uut(
   .Clock(Clock),
   .Reset(Reset),
   .Stop(Stop),
   .InPortData(InPortData),
   .Run(Run),

   .IR(IR),
   .PC(PC),
   .MAR(MAR),
   .MDR(MDR),

   .R0(R0), .R1(R1), .R2(R2), .R3(R3),
   .R4(R4), .R5(R5), .R6(R6), .R7(R7),
   .R8(R8), .R9(R9), .R10(R10), .R11(R11),
```

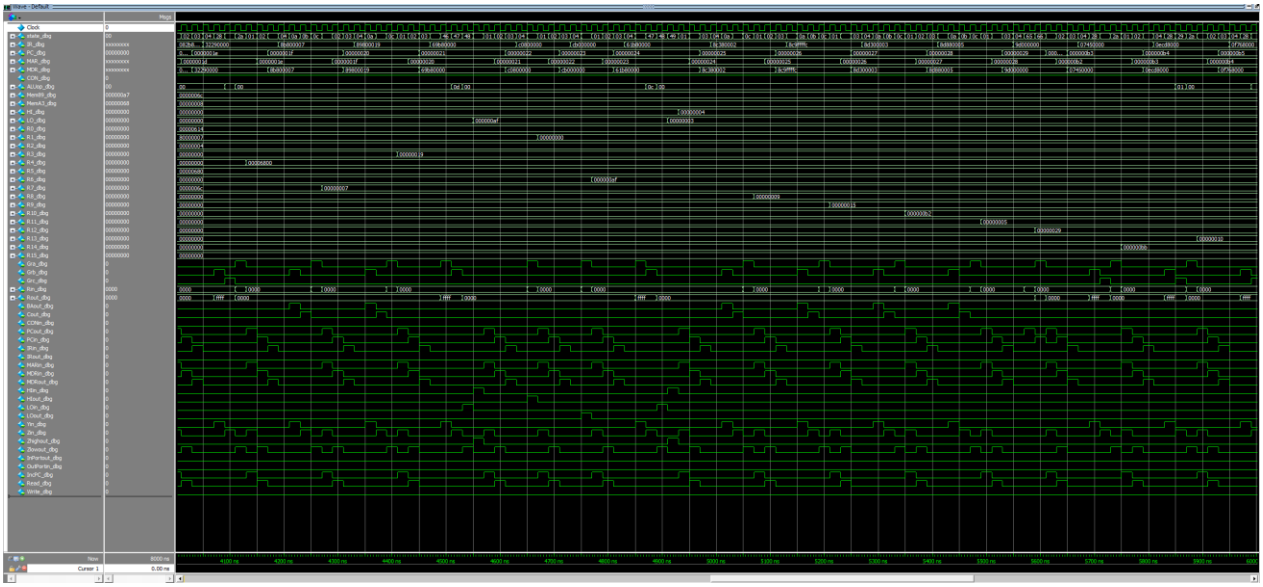
```
.R12(R12), .R13(R13), .R14(R14), .R15(R15),

.HI(HI),
.LO(LO),
.Zhigh(Zhigh),
.Zlow(Zlow),

.mem_89(mem_89),
.mem_A3(mem_A3),

.PCout_dbg(PCout_dbg),
.Zhighout_dbg(Zhighout_dbg),
.Zlowout_dbg(Zlowout_dbg),
.MDRout_dbg(MDRout_dbg),
.MARin_dbg(MARin_dbg),
.PCin_dbg(PCin_dbg),
.MDRin_dbg(MDRin_dbg),
.IRin_dbg(IRin_dbg),
.Yin_dbg(Yin_dbg),
.Zin_dbg(Zin_dbg),
.HIin_dbg(HIin_dbg),
.LOin_dbg(LOin_dbg),
.Cout_dbg(Cout_dbg),
.InPortout_dbg(InPortout_dbg),
.OutPortin_dbg(OutPortin_dbg),
.Read_dbg(Read_dbg),
.Write_dbg(Write_dbg),
```

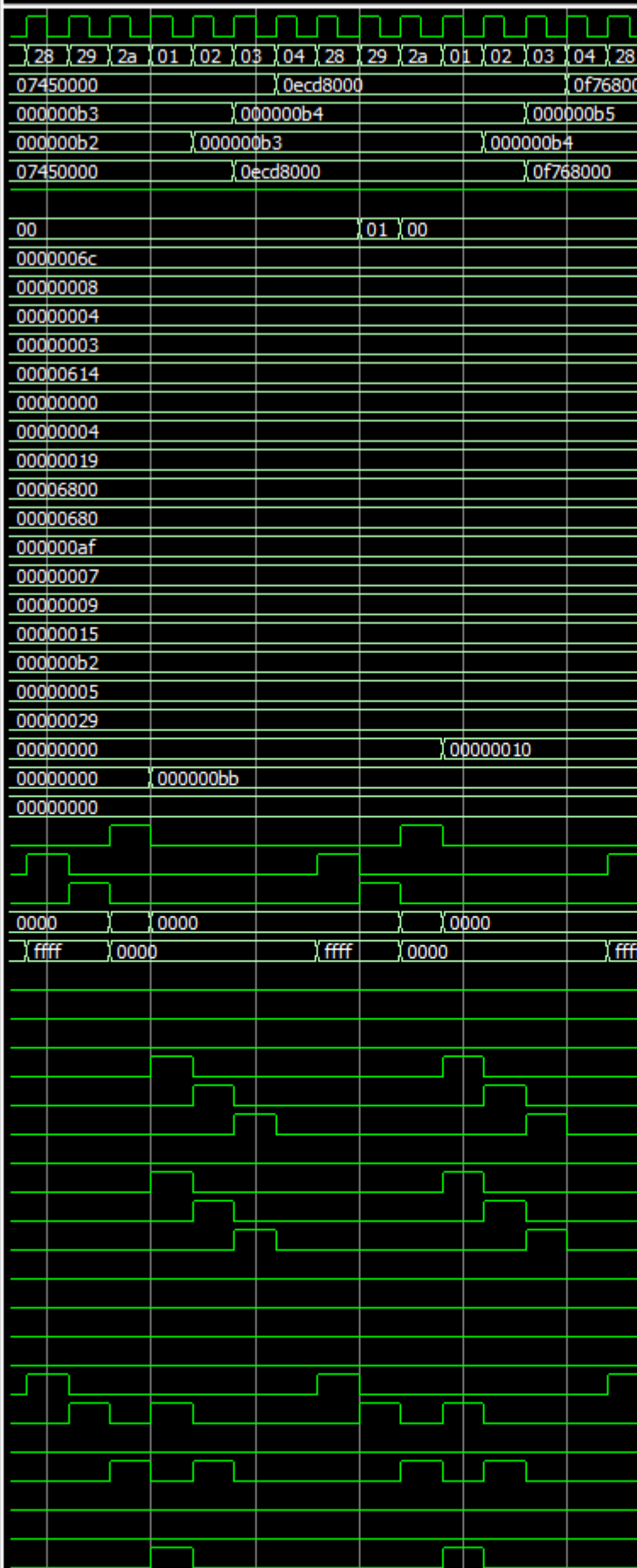
```
    .ALUop_dbg(ALUop_dbg)
);


initial begin
    Clock = 1'b0;
    forever #10 Clock = ~Clock;
end


initial begin
    Reset = 1'b1;
    Stop = 1'b0;
    InPortData = 32'b0;
    #30 Reset = 1'b0;
    #5000;
end


endmodule
```

Simulation







Wave - Default

	Msgs
Clock	1
state_dbg	02
IR_dbg	89800019
PC_dbg	00000020
MAR_dbg	00000020
MDR_dbg	89800019
CON_dbg	1
ALUop_dbg	00
Mem89_dbg	0000006c
MemA3_dbg	00000008
HI_dbg	00000000
LO_dbg	00000000
R0_dbg	00000614
R1_dbg	80000007
R2_dbg	00000004
R3_dbg	00000019
R4_dbg	00006800
R5_dbg	00000680
R6_dbg	00000000
R7_dbg	00000007
R8_dbg	00000000
R9_dbg	00000000
R10_dbg	00000000
R11_dbg	00000000
R12_dbg	00000000
R13_dbg	00000000
R14_dbg	00000000
R15_dbg	00000000
Gra_dbg	0
Grb_dbg	0
Grc_dbg	0
Rin_dbg	0000
Rout_dbg	0000
BAout_dbg	0
Cout_dbg	0
CONin_dbg	0
PCout_dbg	0
PCin_dbg	1
IRin_dbg	0
IRout_dbg	0
MARin_dbg	0
MDRin_dbg	1
MDRout_dbg	0
HIin_dbg	0
HIout_dbg	0
LOin_dbg	0
LOout_dbg	0
Yin_dbg	0
Zin_dbg	0
Zhighout_dbg	0
Zlowout_dbg	1
InPortout_dbg	0
OutPortin_dbg	0
IncPC_dbg	0